Spectral embedding para descripción de imágenes

Matemáticas para el Aprendizaje Automático, 2025/2026

Hannah Macadam, Pablo Pérez, Ane Redondo, David Á. Valenzuela

Universidad Autónoma de Madrid

24 de marzo de 2026

Índice de contenidos

1. Objetivos
2. Metodología
3. Fundamentos teóricos
4. Datos
   - Descripción del dataset
   - Preparación de los datos
5. Implementación
   - Elección del hiperparámetro en la construcción del grafo
   - Construcción del grafo y de la matriz de afinidad
   - Detalles del *embedding* resultante
   - Relación con el *spectral clustering*
6. Conclusiones

Objetivos

Objetivo principal

Aplicar *spectral embedding* para reducir la dimensionalidad y comprender la estructura del dataset Fashion MNIST.

Objetivos específicos:

- Preparar los datos.
- Seleccionar los hiperparámetros óptimos.
- Construir el grafo de vecindad bajo distintas métricas y matrices de pesos.
- Determinar la dimensión óptima del *embedding*.
- Explorar *clustering* con KMeans sobre la variedad obtenida.

Metodología

Herramientas:

- LLMs: Claude, Gemini, ChatGPT
- Entorno de trabajo: Google Colab (Python)
- Presentación en LaTex utilizando Beamer

Plan de trabajo:

- Tutorías
- Reuniones en equipo
- Trabajo individual

Fundamentos teóricos

Os hablaremos de una técnica de aprendizaje no supervisado, basada en el aprendizaje de variedades.

Spectral embedding

Método de reducción de la dimensionalidad no lineal, que preserva la estructura geométrica local de los datos.

Sean los datos $\{x_i\}_{i=1}^m \subset \mathbb{R}^n$ y la dimensión objetivo $k \ll n$.

Pasos:

1: *Grafo de vecindad:* conectar puntos próximos (e.g. k-NN, ϵ-grafo)
2: *Pesos:* asignar $w_{ij} \geq 0$ a cada arista (e.g. binario, núcleo RBF)
3: *Laplaciano:* $L = D - W, \quad d_i = \sum_j w_{ij}$
4: *Laplaciano normalizado:* $M = D^{-1/2} L\, D^{-1/2}$
5: *Autovectores:* $g_2, \ldots, g_{k+1}$ de M con $0 = \lambda_1 \leq \lambda_2 \leq \cdots$
6: *Embedding:* $x_i \mapsto (g_2(i), \ldots, g_{k+1}(i)) \in \mathbb{R}^k$

Fundamentos teóricos

Dado $L = D - W$,

$$f^\top L f = \frac{1}{2} \sum_{i,j} w_{ij} (f_i - f_j)^2$$

Minimizar $f^\top L f$ sujeto a $f^\top D f = 1$:

$$\min_f \frac{f^\top L f}{f^\top D f}$$

Con $g = D^{1/2} f$:

$$\frac{f^\top L f}{f^\top D f} = \frac{g^\top M g}{\|g\|^2}, \qquad M = D^{-1/2} L D^{-1/2}$$

$$\min_g \frac{g^\top M g}{\|g\|^2} = \lambda_2, \qquad \text{alcanzado en } g_2$$

Descripción del dataset

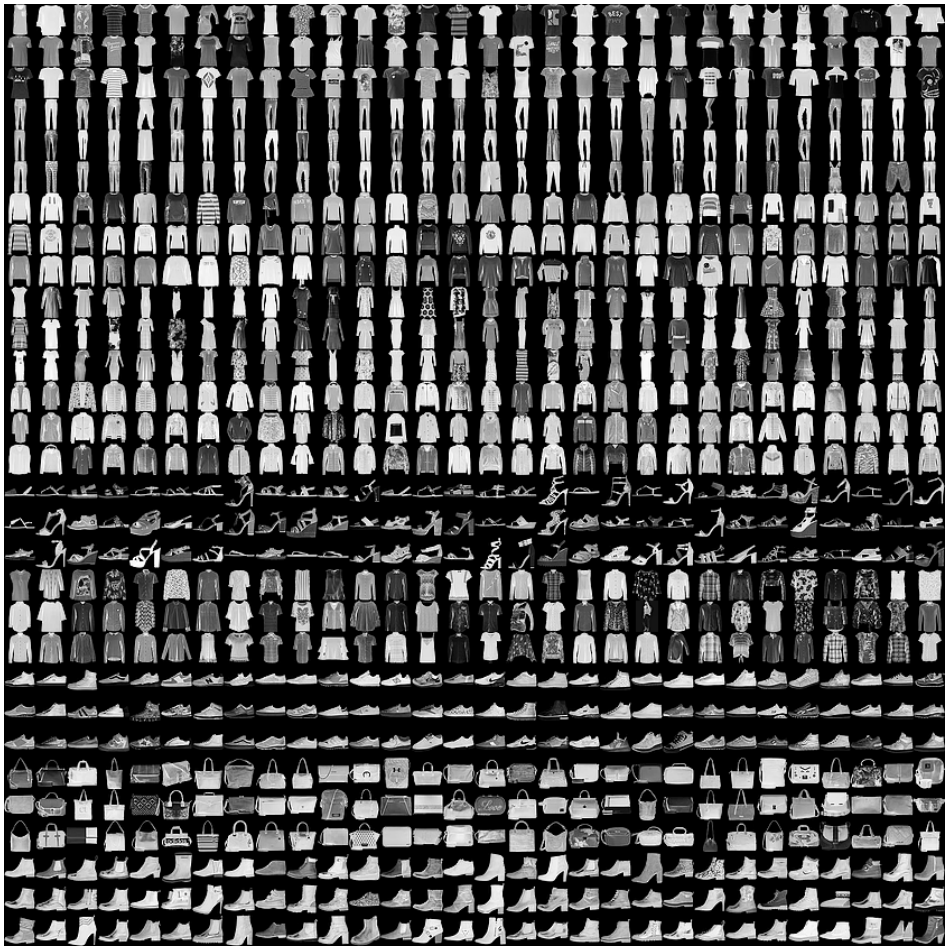


Figura: Fashion MNIST

Descripción del dataset

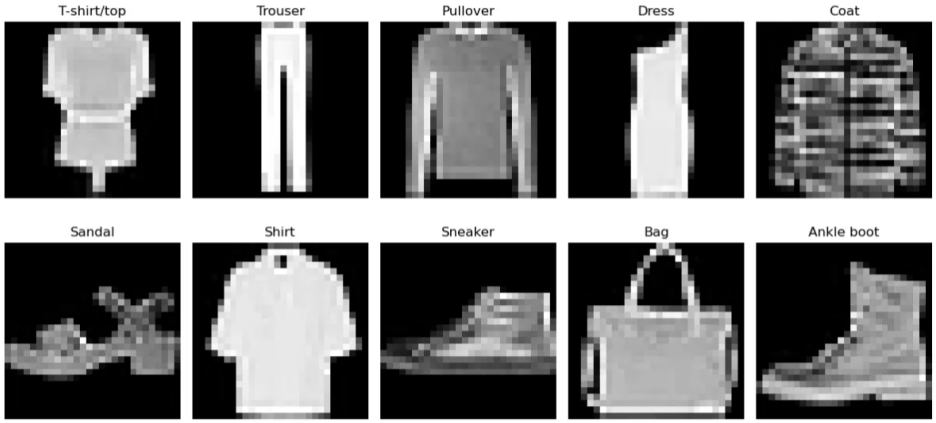


Figura: Fashion MNIST

Características:

- 10 clases (camisetas, abrigos, pantalones, etc.).
- Tamaño del dataset: 70.000 imágenes balanceadas entre clases.
- Cada imagen tiene 28x28 píxeles en escala de grises, i.e., es un vector de píxeles en $\mathbb{R}^{784}$.

Preparación de los datos

Tras cargar las imágenes como vectores de píxeles, nos aseguramos de que cada píxel pase de tomar valores en [0,255] a tomarlos en [0,1].

```
# fetch_openml descarga datasets de un repositorio publico, a partir de un id unico del
    mismo
from sklearn.datasets import fetch_openml

# Cargamos el dataset Fashion-MNIST
# return_X_y hace que esta funcion devuelva en X la matriz cuyas filas son los vectores
    de pixeles y en y el vector de labels
# Esta funcion nos devuelve cada imagen como un vector de IR^784, en forma de numpy array
X, y = fetch_openml("Fashion-MNIST", version=1, return_X_y=True, as_frame=False)

# Hacemos que los pixeles tomen valores en [0,1]
X = X / 255.0
```

Spoiler: dos nociones de distancia

Deben capturar la esencia de los datos.

Distancia euclídea

$$d(x, y) = \sqrt{\sum_{i=1}^{784} (x_i - y_i)^2}$$

Distancia coseno

$$d_{cos}(x, y) = 1 - \frac{\sum_{i=1}^{n} x_i y_i}{\sqrt{\sum_{i=1}^{n} x_i^2} \sqrt{\sum_{i=1}^{n} y_i^2}}$$

La distancia coseno puede sernos de utilidad si nos interesa más la orientación que la magnitud de los vectores.

¿Qué hiperparámetro interviene en el *spectral embedding*?

Las imágenes son vectores de $\mathbb{R}^{784}$, pero buscamos una variedad no lineal de dimensión *pequeña* que explique bien la estructura de los datos. Esto implica que datos cercanos en el espacio original tienen que estar próximos en la variedad.

¿Cómo lo hacemos?

- Grafo k-NN (también podríamos utilizar un ε-graph).
- Probando las dos distancias explicadas, lo que implica elegir el hiperparámetro k más adecuado en cada caso.

Elección de hiperparámetro en la construcción del grafo

```
from sklearn.neighbors import NearestNeighbors

# Valores de k que vamos a estudiar
k_values = [1, 2, 3, 4, 5, 10, 15, 20, 25, 30, 35, 40, 45, 50, 60, 70, 85, 100]

# Tomamos el mayor valor de k para calcular todos los vecinos de una vez
k_max = max(k_values)

# Vecinos mas cercanos con distancia euclidea
nn_eu = NearestNeighbors(n_neighbors=k_max, metric="euclidean")
# Ajustamos los datos al modelo
nn_eu.fit(X)

# Obtenemos las distancias y los indices de los vecinos mas proximos con la distancia
    euclidea. dist_eu, ind_eu son matrices de tamanio 70.000x100
dist_eu, ind_eu = nn_eu.kneighbors(X)

# Vecinos mas cercanos con distancia coseno
nn_co = NearestNeighbors(n_neighbors=k_max, metric="cosine")
# Ajustamos los datos al modelo
nn_co.fit(X)

# Obtenemos las distancias y los indices de los vecinos mas proximos con la distancia
    coseno
dist_co, ind_co = nn_co.kneighbors(X)
```

Elección del hiperparámetro en la construcción del grafo

Usando las matrices `dist_eu` y `dist_co` podemos graficar unos histogramas, para así identificar *buenos* valores de k en cada caso.

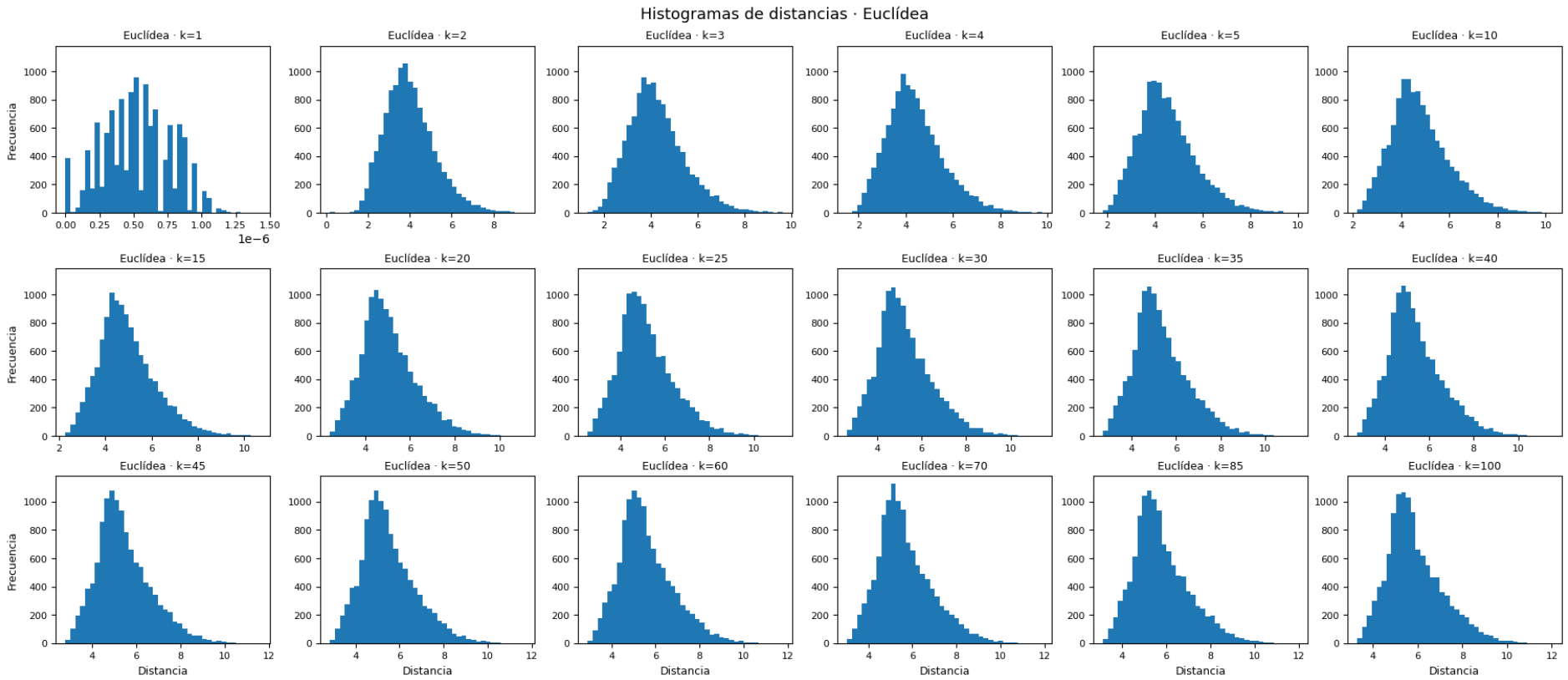


Figura: Histogramas para la elección del k con la distancia euclídea

Elección del hiperparámetro en la construcción del grafo

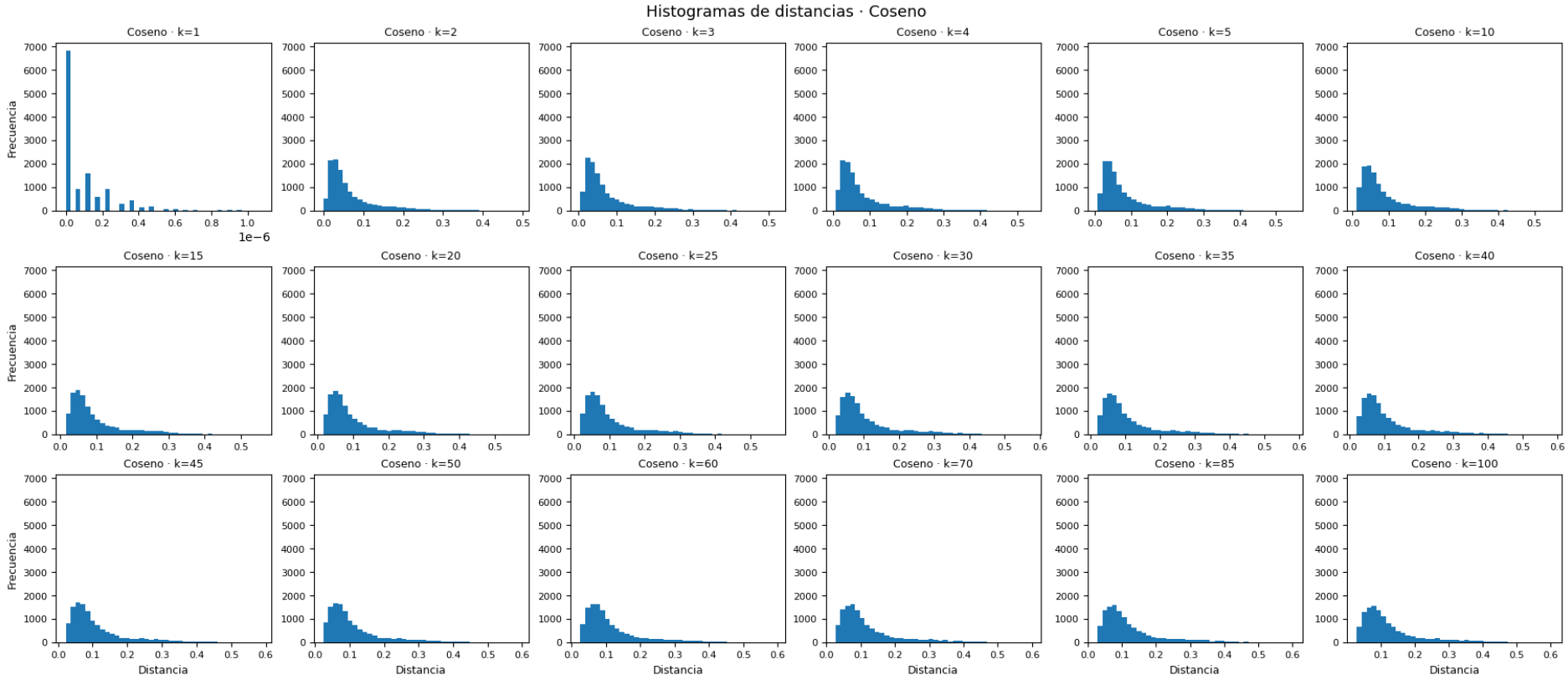


Figura: Histogramas para la elección del k con la distancia coseno

Conclusión. k=10 para la distancia euclídea y k=15 para la distancia coseno parecen razonables.

Varias maneras de construir el grafo y la matriz de afinidad

Aún nos queda elegir cómo construir la matriz con pesos (matriz de afinidad) W y decantarnos por una de las distancias.

Opciones consideradas:

1. k-NN usando distancia euclídea y pesos binarios.
2. k-NN usando distancia euclídea y pesos RBF (gaussiano).
3. W dada de forma densa por RBF (gaussiano).
4. W dada de forma densa por RBF (defecto de `sklearn`).
5. k-NN usando distancia coseno y pesos binarios.
6. k-NN usando distancia coseno y pesos RBF.

Nota. A partir de este punto, trabajamos con un subconjunto de 12.000 imágenes (balanceadas entre clases) del dataset.

k-NN, distancia euclídea

De momento, imponemos que la dimensión de la variedad sea 2.

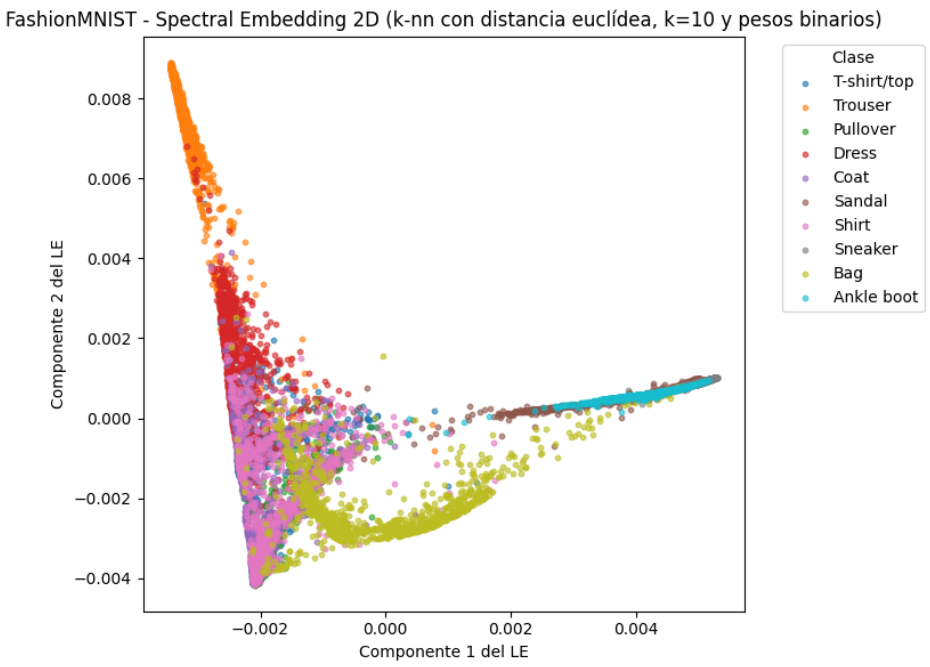
```
# Importamos el algoritmo de aprendizaje no supervisado que implementa el spectral
    embedding
from sklearn.manifold import SpectralEmbedding

# Valor optimo del hiperparametro para k-nn con la distancia euclidea
n_neighbors = 10

# Todo por defecto. La matriz de afinidad es una matriz 'sparse' calculada usando k-nn
    con la distancia euclidea
se1 = SpectralEmbedding(n_components=2, affinity='nearest_neighbors',
                        n_neighbors=n_neighbors)
# Ajusta el subconjunto de datos de 12.000 imagenes al modelo y devuelve X1, que es el
    spectral embedding resultante
X1 = se1.fit_transform(X)

representacion_scatter(X1, y, "FashionMNIST - Spectral Embedding 2D (k-nn con distancia
    euclidea, k=10 y pesos binarios)")
```

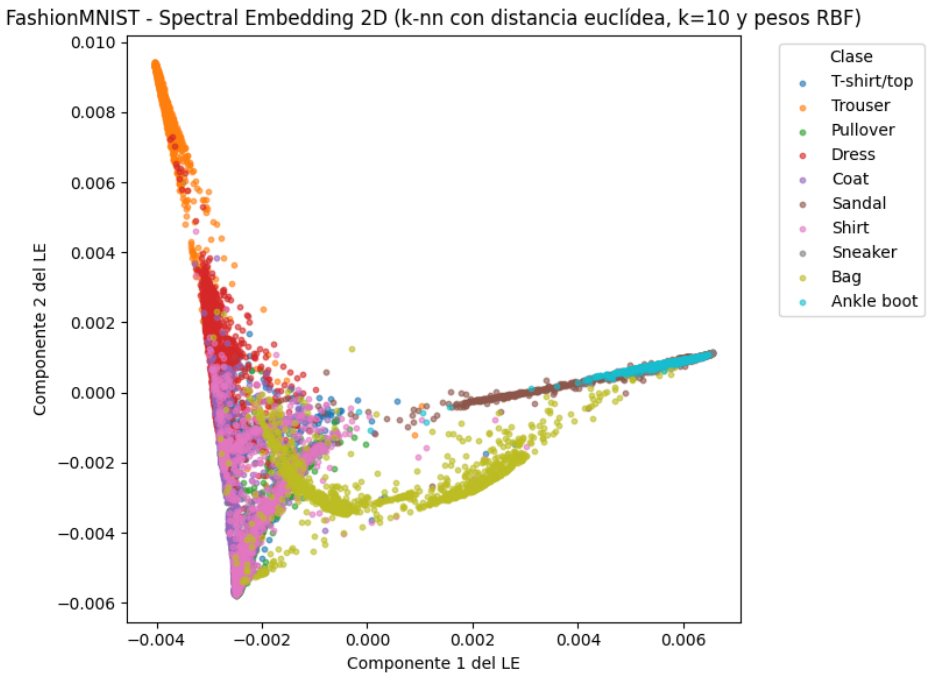
k-NN, distancia euclídea

Cuando pedimos que los pesos vengan determinados por el kernel RBF gaussiano, hay que precomputar 'a mano' la matriz de afinidad.

```
# Calculamos sigma como la mediana de las distancias euclideas de cada dato a sus 10
    vecinos mas cercanos
sigma = np.median(dist_eu[:, n_neighbors-1])

# Definimos gamma a partir de sigma
gamma = 1 / (2 * sigma**2)

from sklearn.neighbors import kneighbors_graph

# Calculo de la matriz de afinidad
# Construimos el grafo k-nn usando como pesos las distancias euclideas entre vecinos
A = kneighbors_graph(X, n_neighbors=n_neighbors, mode='distance')

# Transformamos esas distancias en afinidades mediante un kernel RBF
A.data = np.exp(-A.data**2 * gamma)
W3 = (A + A.T)  # simetrizar sumando
W3.data /= 2    # promedio donde se solapan (altera algunas distancias)

# Definimos el modelo utilizando la matriz de afinidad que hemos construido
se3 = SpectralEmbedding(n_components=2, affinity='precomputed')

# Ajustamos los datos para obtener la variedad resultante
X3 = se3.fit_transform(W3)

representacion_scatter(X3, y, "FashionMNIST - Spectral Embedding 2D (k-nn con distancia
    euclidea, k=10 y pesos RBF)")
```

k-NN, distancia euclídea





¿Os llama algo la atención?

W densa con RBF

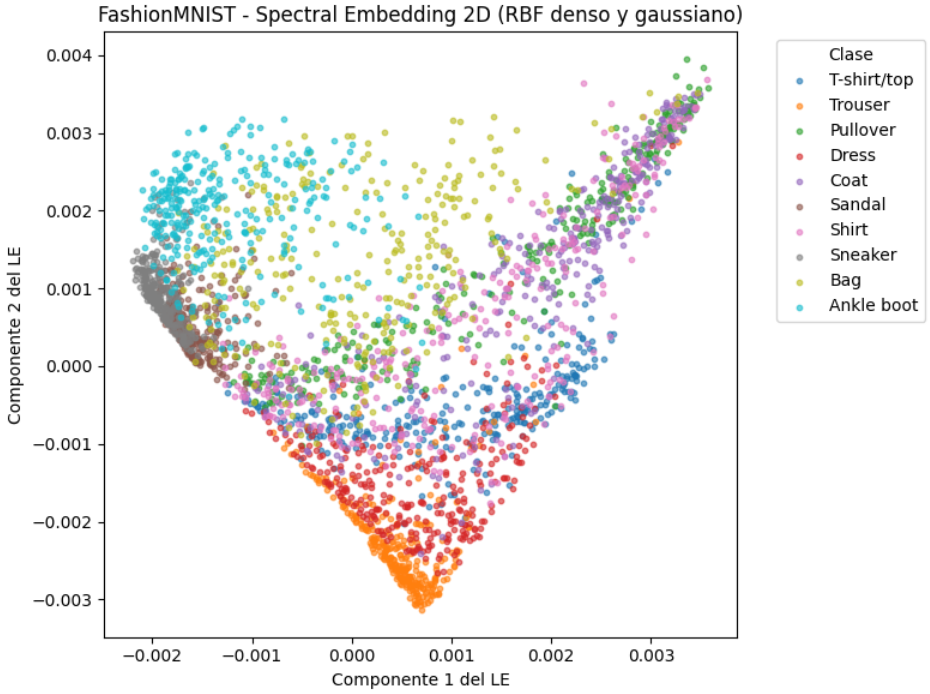
Otra posibilidad es definir directamente la matriz de afinidad como $W_{ij} = exp(-\gamma \parallel x_i - x_j \parallel^2)$. ¿Qué γ utilizamos?

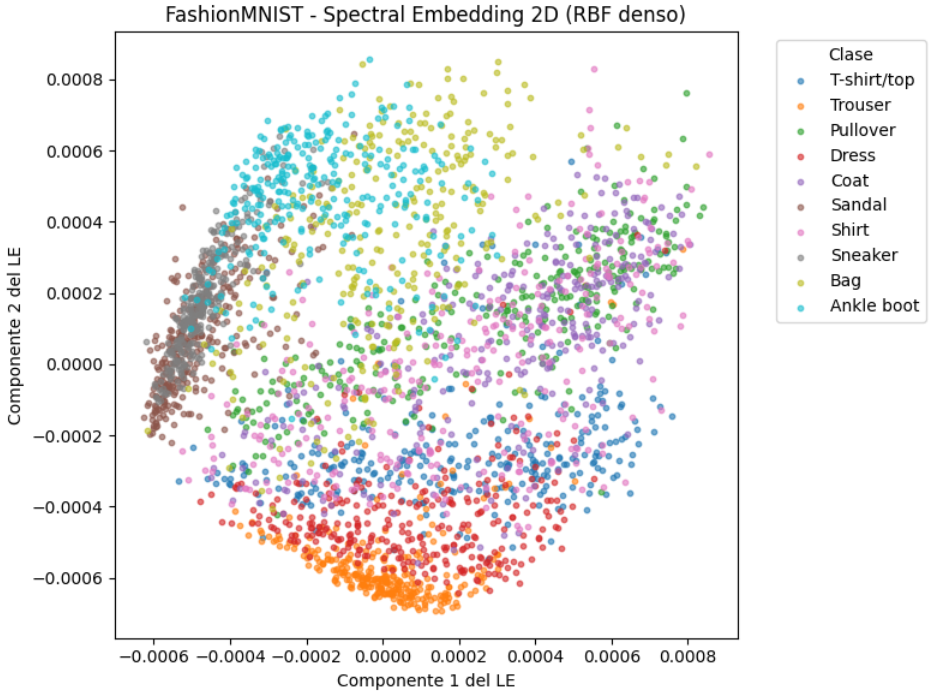
En el siguiente código lo elegimos de forma que el núcleo RBF sea gaussiano, pero hay más opciones.

```
# Inicializamos el modelo utilizando el RBF con parametro gamma como hemos
# descrito arriba
se2 = SpectralEmbedding(n_components=2, affinity='rbf', gamma=gamma)

# Ajustamos el subconjunto deimagenes al modelo y obtenemos X2, que es la
# variedad de dimension 2 que mejor representa la estructura local no lineal de
# dichas imagenes
X2 = se2.fit_transform(X_rbf)

representacion_scatter(X2, y_rbf, "FashionMNIST - Spectral Embedding 2D (RBF denso y
     gaussiano)")
```

W densa con RBF

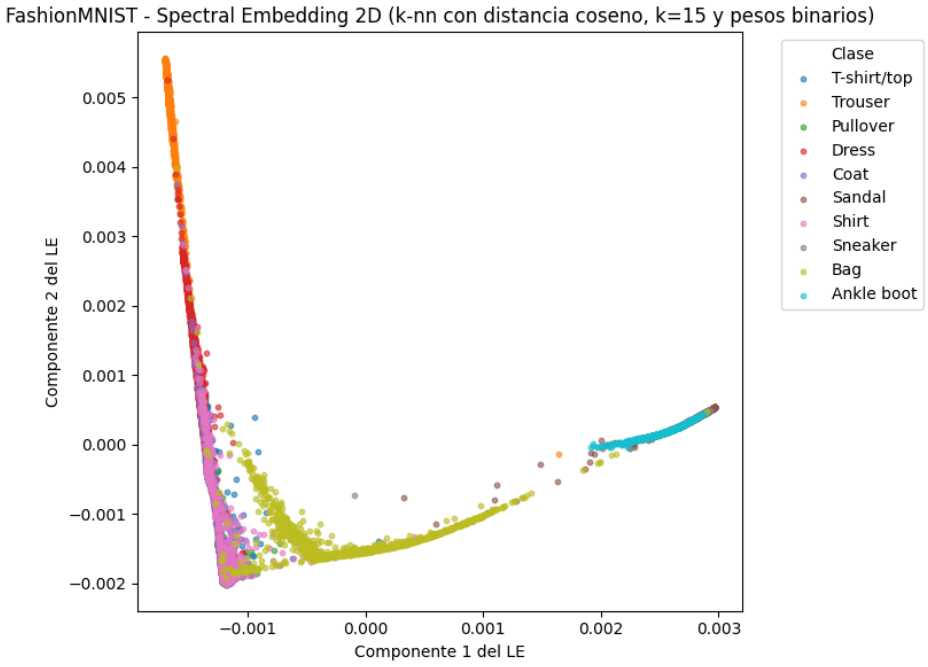
A la izquierda, la variedad 2D obtenida tras ejecutar el código anterior. A la derecha, elegimos el γ por defecto de `sklearn`.

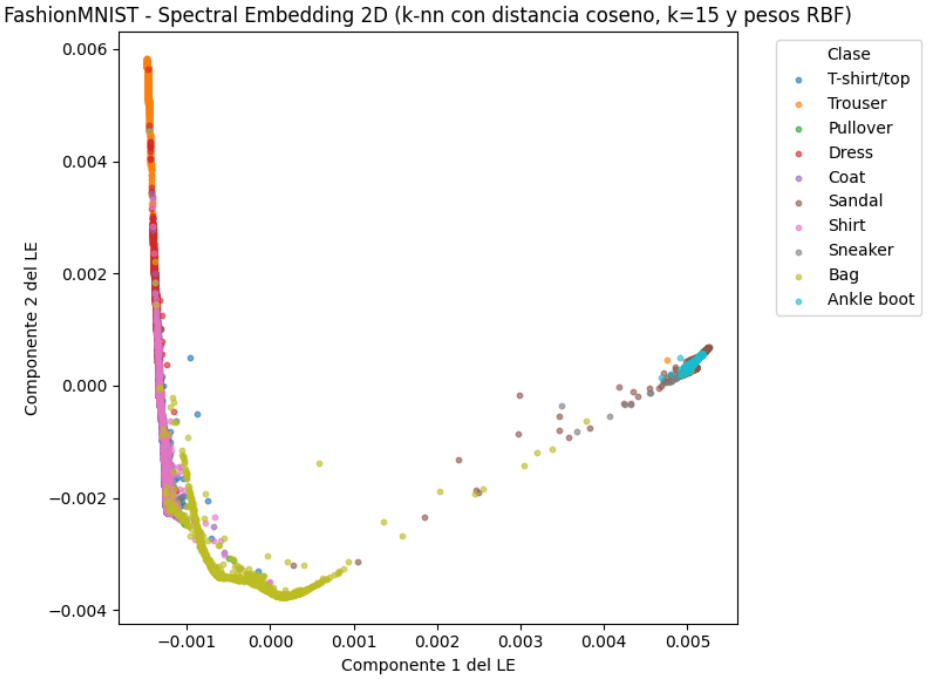




k-NN, distancia coseno

La matriz de afinidad también se puede precomputar para la distancia coseno, aunque los resultados obtenidos no son tan visuales como nos gustaría.





Nuestra elección

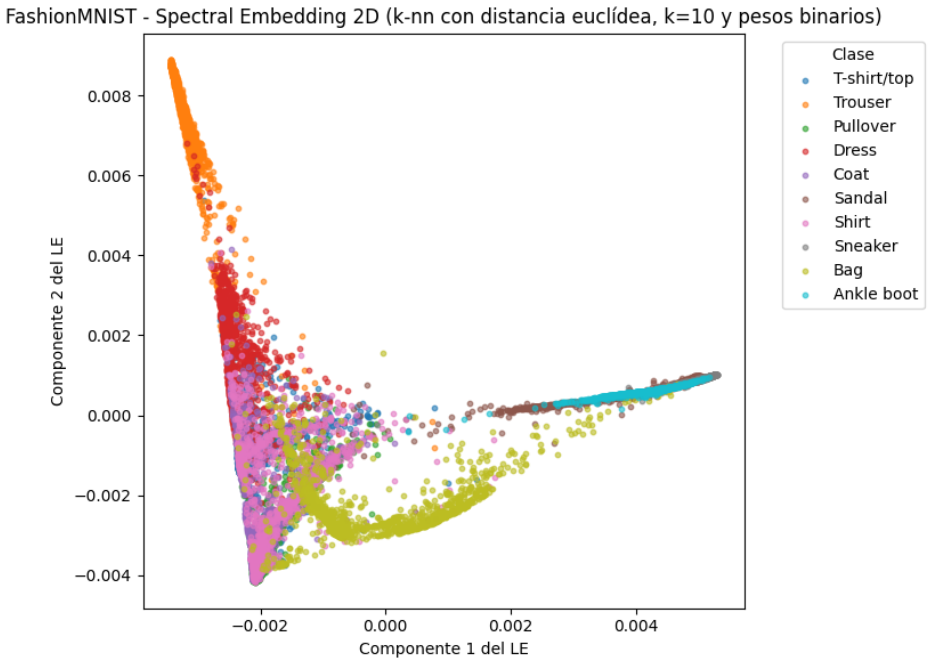
En base a la discusión anterior, vamos a construir el grafo utilizando k-NN (k=10) con la distancia euclídea, definiendo la matriz de afinidad mediante pesos binarios.

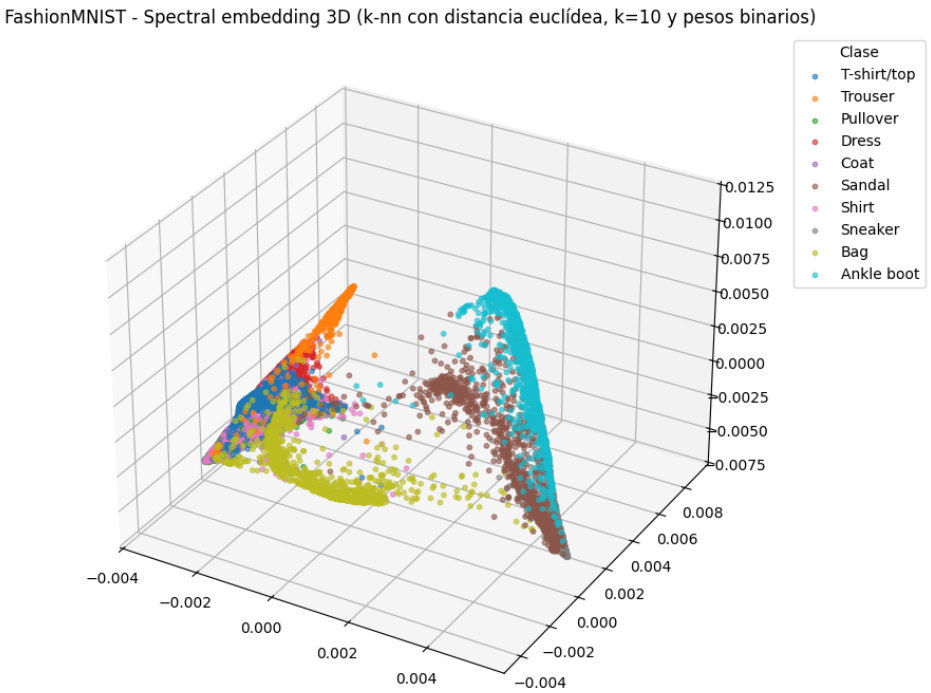
Justificación:

- Separa más las clases (e.g., bolsos, pantalones, calzado, prendas de arriba).
- Sencillez: los pesos binarios son más fáciles de calcular que la RBF.

Visualización en 2D y 3D del embedding

Además de visualizar el *spectral embedding* obtenido en 2D, veamos qué sale en 3D.





Interpretación gráfica

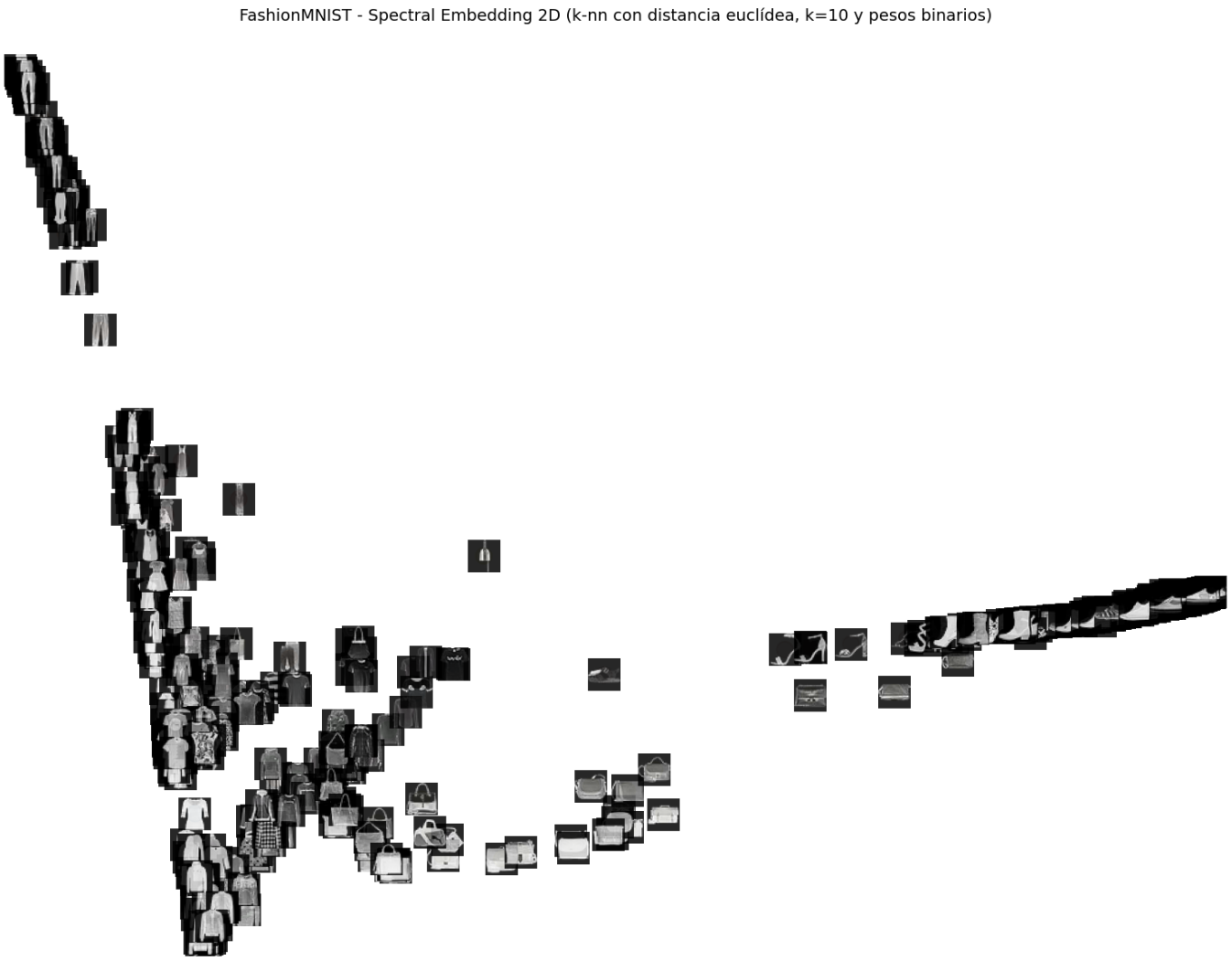


Figura: Muestra de imágenes en el *embedding*

Dimensión óptima del embedding

Hasta ahora, solo hemos trabajado con las variedades obtenidas en 2D y 3D. En este punto, nos interesamos por saber si existe alguna dimensión *pequeña* que explique mejor la estructura de los datos. Para ello, representamos el espectro del Laplaciano.

```
# Para calcular autovalores y autovectores de matrices simetricas sparse
from scipy.sparse.linalg import eigsh
# Para construir el laplaciano a partir de la matriz de afinidad
from scipy.sparse.csgraph import laplacian

# Creamos el modelo con mas componentes de los utilizados hasta ahora
se = SpectralEmbedding(n_components=20)
# Ajustamos el modelo a los datos
se.fit(X)

# Los autovalores se calculan a partir de la matriz de afinidad construida
W = se.affinity_matrix_  # matriz de afinidad
# Construccion del laplaciano normalizado
L = laplacian(W, normed=True)

# eigsh calcula los 20 autovalores mas pequenos en magnitud de L (Small Magnitude)
vals, _ = eigsh(L, k=20, which='SM')
# Los ordenamos de menor a mayor
vals = np.sort(vals)

plt.plot(range(len(vals)), vals, marker='o')
plt.xticks(range(len(vals)))
# Omitimos detalles menores de los plots
```

Dimensión óptima del embedding

¿Dónde se observa el codo del espectro?

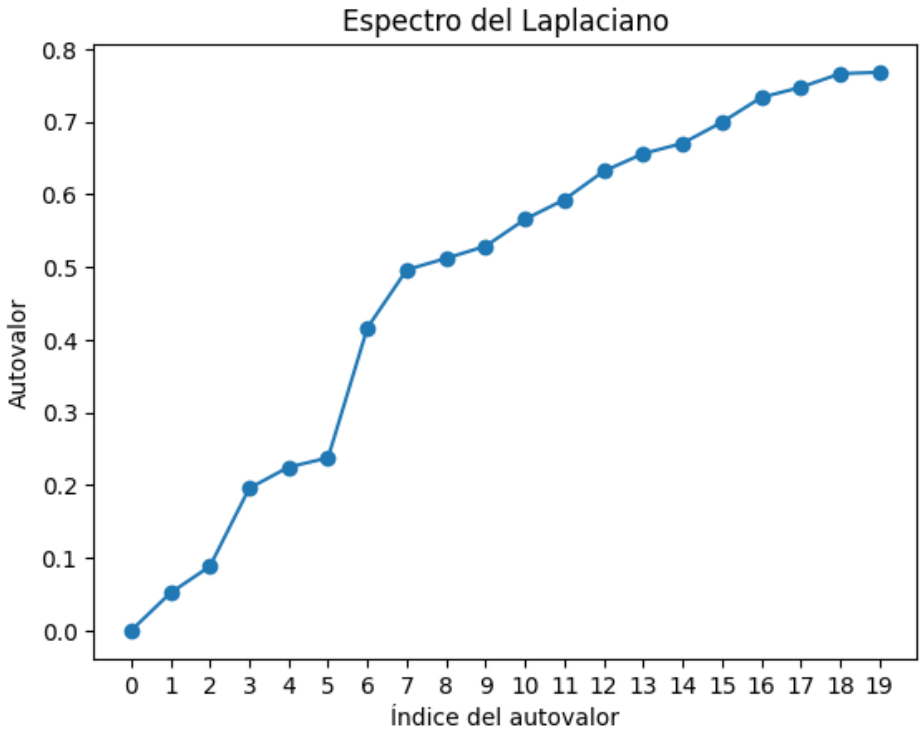


Figura: Espectro del Laplaciano del grafo

Conclusión. Parece que una variedad de dimensión 5 explica *bien* la mayor parte de los datos.

Embedding y *clustering*: dos caras de la misma moneda

Mediante el algoritmo de *spectral embedding* hemos reducido la dimensionalidad de los datos, llevándolos de 784 dimensiones a 2, 3 ó 5.

Este tipo de técnicas se conocen como soft-clustering, luego resulta natural aplicar algoritmos empíricos de agrupamiento (como KMeans) para hacer un hard-clustering. Por tanto, nuestro objetivo ahora es etiquetar los datos en las variedades obtenidas.

Pero, ¿en qué consiste este algoritmo?

K-means

Algoritmo de clustering que particiona los datos en K clústeres, asignando cada punto al centroide más cercano y actualizando después dichos centroides para minimizar la varianza dentro de cada clúster.

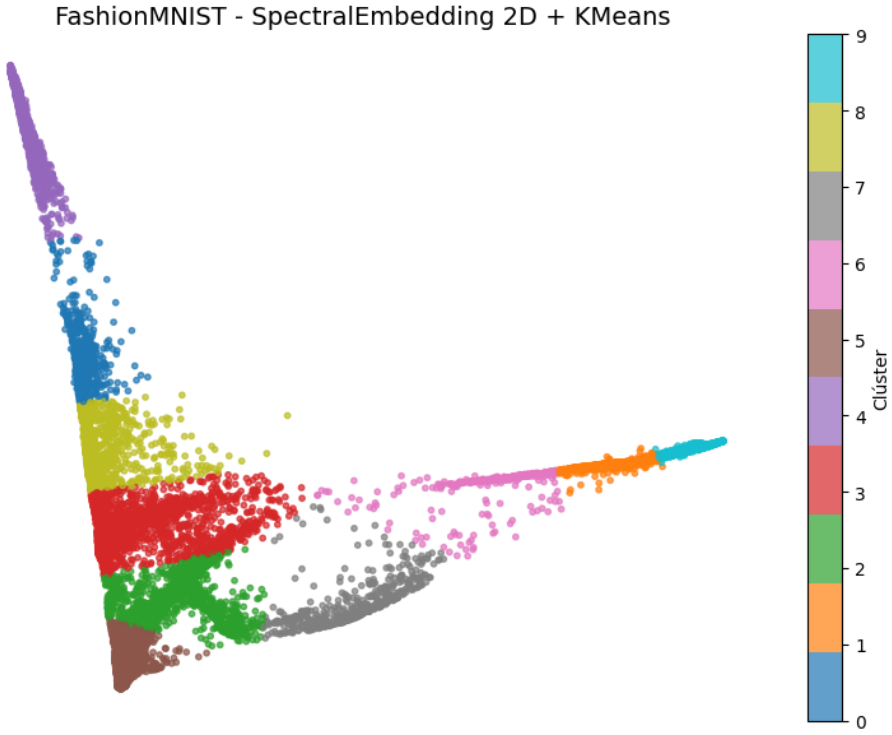
Clústeres resultantes con KMeans (K=10)

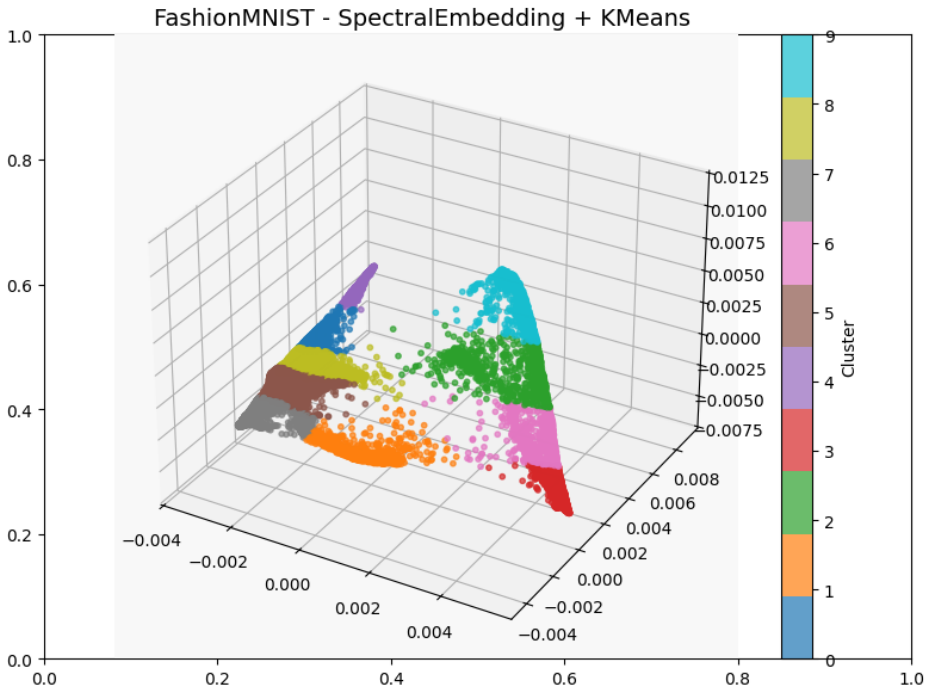
Por ahora, ejecutamos el algoritmo KMeans con K=10 clústeres y centroides. Esto se debe a que el dataset Fashion MNIST cuenta con 10 clases, luego nos interesa ver cómo de bien se agrupan las prendas en las variedades.

```
from sklearn.cluster import KMeans

N_CLUSTERS = 10  # FashionMNIST tiene 10 clases

# Aplica el algoritmo kmeans para clustering
# n_clusters determina el numero de clusteres y centroides a generar
# random_state inicializa los centroides de forma determinista
kmeans = KMeans(n_clusters=N_CLUSTERS, random_state=RANDOM_STATE)

# fit ajusta el modelo kmeans a la variedad 2d
# predict asigna a cada dato una etiqueta de clase
cluster_labels = kmeans.fit_predict(X1_2d)

fig, ax = plt.subplots(figsize=(8, 6))
ax.set_facecolor("#f8f8f8")

# Recorremos todos los puntos de la variedad
idx = range(len(X1))

# Cada punto se pinta del color de su cluster
scatter = ax.scatter(X1[idx, 0], X1[idx, 1], c=cluster_labels[idx], cmap="tab10", s=10,
     alpha=0.7)

# Omitimos detalles menores de la grafica
```

Clústeres resultantes con KMeans (K=10)

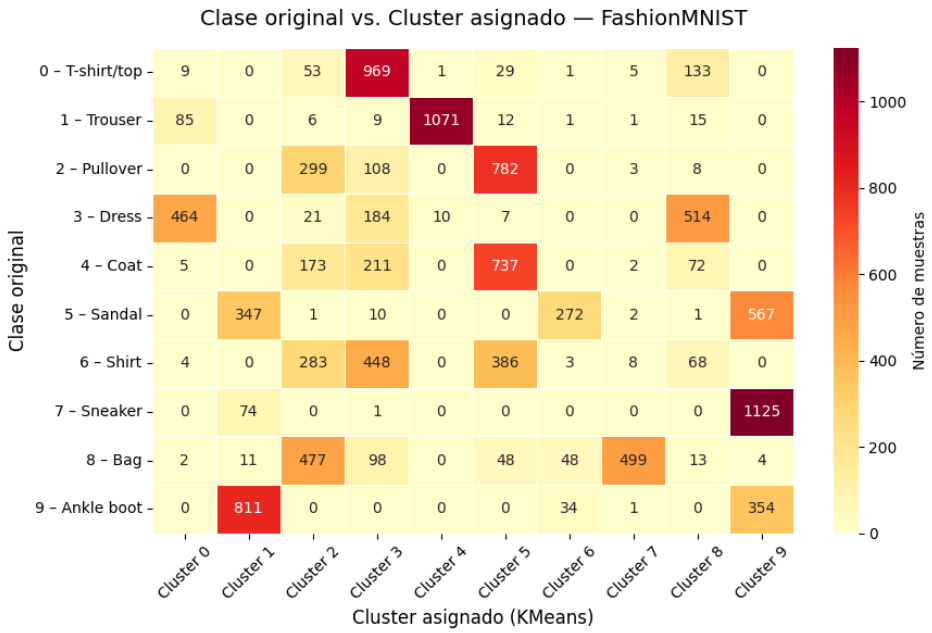
Veamos los clústeres que forma el algoritmo en las variedades 2D y 3D.

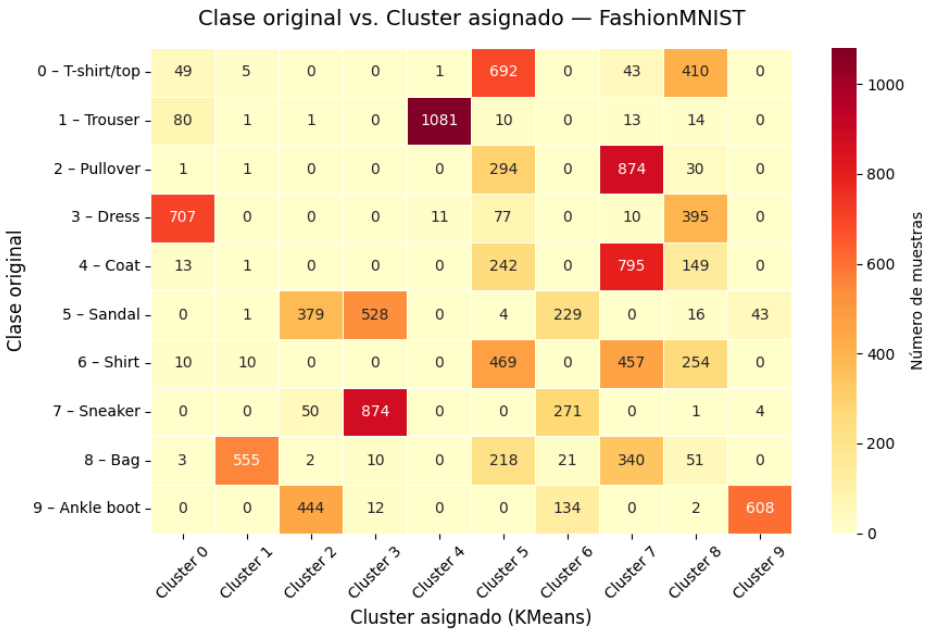




Matrices de confusión

Puede resultar interesante utilizar matrices de confusión para ver qué prendas se agrupan en cada uno de los clústeres. Primero, visualizamos los correspondientes a las variedades 2D y 3D.





Matrices de confusión

¿Qué prendas se asignan a cada clúster en la variedad 5D? ¿Tienen sentido los grupos formados?

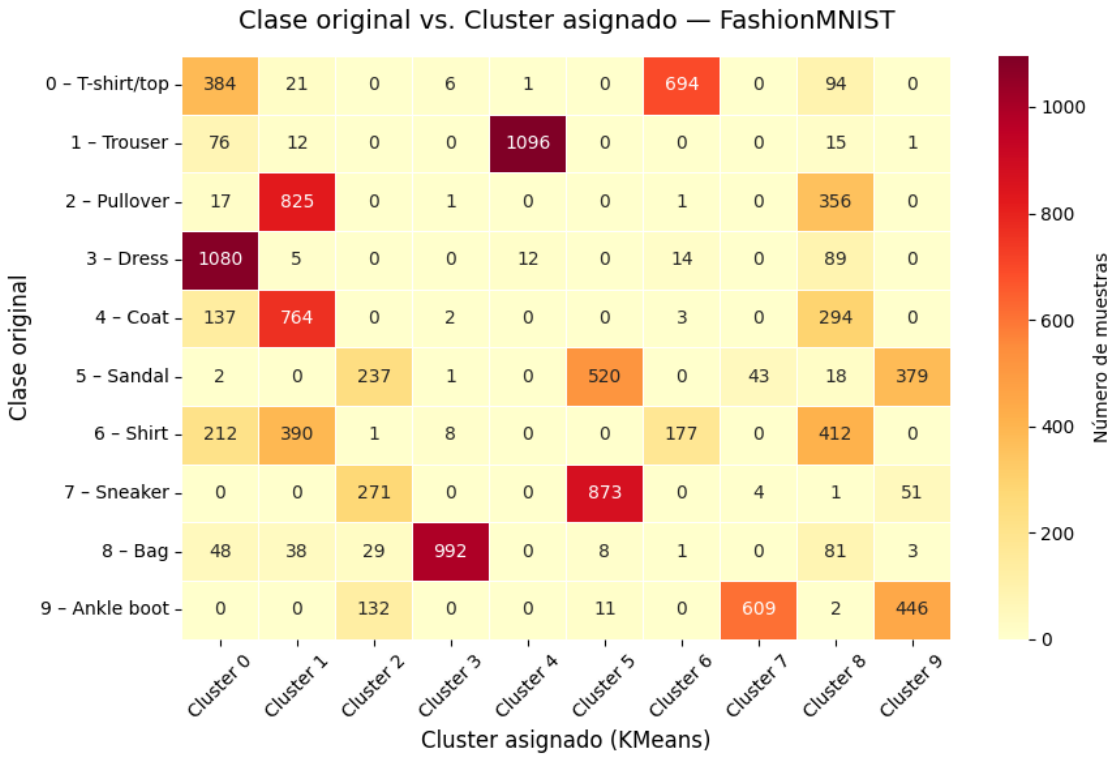


Figura: Matriz de confusión asociada a la variedad 5D

Elección del hiperparámetro K en KMeans (una breve idea)

Puede ser conveniente formar menos grupos, de modo que haya menos prendas entremezcladas entre varios clústeres. Para ello, tenemos que elegir un valor del hiperparámetro K adecuado.

Existen varios criterios para elegir dicho valor. Nosotros usamos dos:

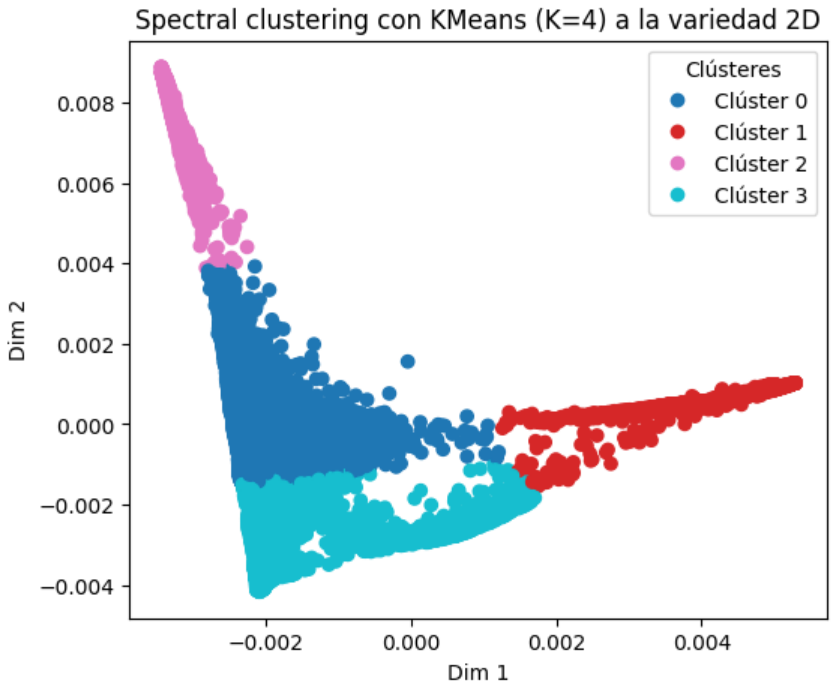
1. Método del codo: representa la suma de las distancias cuadradas de cada punto a su centroide. Elegimos un K a partir del cual esta cantidad deja de disminuir de forma apreciable.
2. Índice de silueta: toma valores entre [-1,1], midiendo la cohesión intraclúster y la separación entre clústeres. Elegimos un K que maximice este índice.

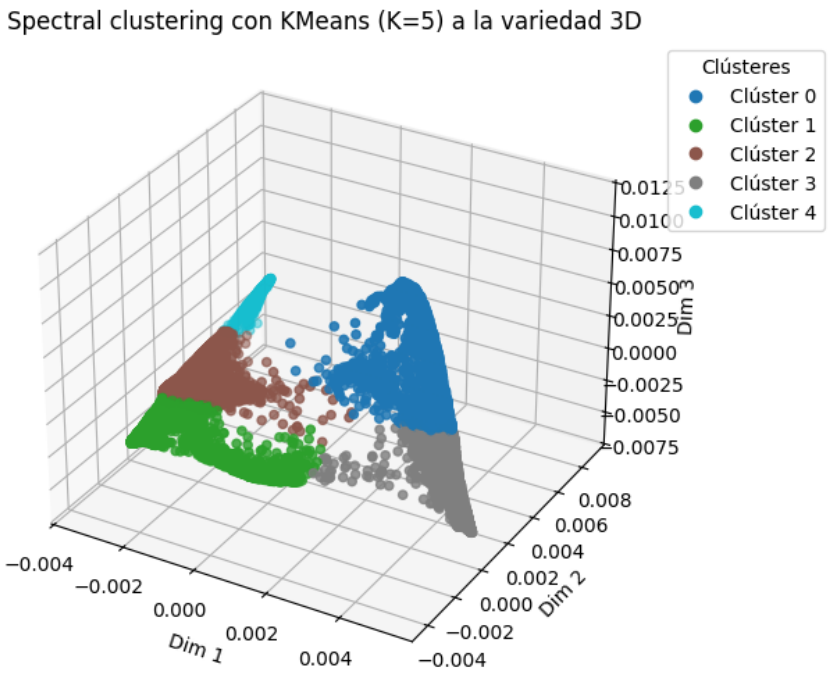
Los resultados se presentan en la siguiente tabla.

K óptimo	**2D**	**3D**	**5D**
Codo	4	5	6
Silueta	3	4	6

Elección del hiperparámetro K en KMeans (una breve idea)

Utilizando los valores que nos da el método del codo, obtenemos los siguientes clústeres.





¿Son satisfactorios los resultados? ¿Qué podemos mejorar?

Resumen y conclusiones

¿Qué hemos hecho?

- Representación de un *dataset* de imágenes: el FashionMNIST.
- Discusión sobre la construcción del grafo: distancias, k, diferentes métodos de construir la matriz de afinidad.
- Visualización e interpretación: el *embedding* calculado encuentra (cierta) estructura en los datos.
- Intento de clusterizar esa estructura: aplicamos KMeans al *embedding*.

Referencias

Papers y artículos

- Belkin, M. y Niyogi, P. *Laplacian Eigenmaps for Dimensionality Reduction and Data Representation*. [PDF]
- Ghojogh, B., Ghodsi, A., Karray, F. y Crowley, M. *Laplacian-Based Dimensionality Reduction Including Spectral Clustering, Laplacian Eigenmap, Locality Preserving Projection, Graph Embedding, and Diffusion Map: Tutorial and Survey*. [arXiv]
- Garreau, D., Jitkrittum, W. y Kanagawa, M. *Large sample analysis of the median heuristic*. [arXiv]
- Koloski, B. et al. *Incremental Graph Construction Enables Robust Spectral Clustering of Texts*. [arXiv]

Repositorios de GitHub

- `zalandoresearch/fashion-mnist`. [GitHub]
- `sidzekrom/spectral-clustering`, archivo `mnistHandwriting.py`. [GitHub]
- `sdimi/handwritten-digits-recognition`, archivo `cluster_mnist.py`. [GitHub]

Blogs sobre ML

- GeeksforGeeks. *Gaussian Kernel*. [link]
- GeeksforGeeks. *Determine the optimal value of K in K-Means Clustering*. [link]

Nuestro notebook de Colab

- Notebook de Google Colab del proyecto. [Colab]

¡Muchas gracias!

¿Preguntas?